SECURITY BRIEFING · 2026 EDITION

AUTH.TXT

API AUTH VS OAUTH IN OPENCLAW

A comparative overview of how OpenClaw secures access to its services.

```
C:\> auth.exe --compare
> loading auth protocols...
> methods: api_key | oauth2
> status: SECURED ✓
```


PRESS [ENTER] TO AUTHENTICATE

NEO-BRUTALIST EDITION

TWO WAYS TO AUTHENTICATE

How OpenClaw secures access to its services.

API AUTHENTICATION

A static key or token sent with every request. The server validates the key, the client gets direct access. Simple, fast, and stateless — but the key never expires until you rotate it manually.

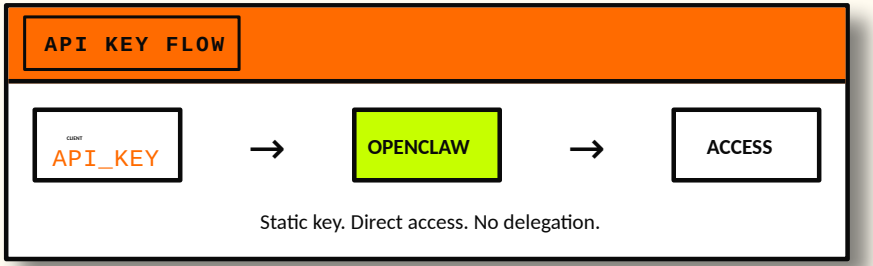
OAUTH AUTHENTICATION

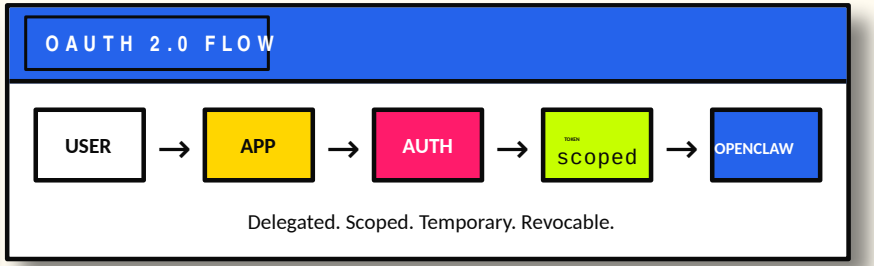
A delegated authorization protocol. Users approve a third-party app to act on their behalf via scoped, temporary access tokens. No shared long-term secrets — tokens expire, scopes limit reach.

FIG.01 · DELEGATED TOKEN EXCHANGE

BOTH METHODS · ONE GOAL · SECURE ACCESS

MECHANISM & SECURITY





ASPECT	API AUTH	OAUTH
KEY TYPE	Static long-lived key	Dynamic short-lived token
ACCESS MODEL	Direct	Delegated
SCOPE	Full key permissions	Scoped per request
EXPIRY	Manual rotation	Auto-expiring + refresh
REVOCATION	Rotate / delete key	Revoke token instantly

USE CASES IN OPENCLAW

Machine-to-machine vs user-to-service — pick the right tool.

API AUTHENTICATION

MACHINE-TO-MACHINE

BEST FOR

- Internal scripts calling OpenClaw APIs from cron jobs
- CI/CD pipelines spinning up agent runs headlessly
- Service-to-service comms inside a trusted VPC
- Personal automation tools with a single owner

WHEN: TRUSTED · INTERNAL · LOW-RISK

OAUTH AUTHENTICATION

USER-TO-SERVICE

BEST FOR

- Third-party apps acting on behalf of a user
- Marketplace integrations with per-user consent
- Web dashboards using Google/GitHub SSO
- Mobile clients needing scoped, revocable access

WHEN: EXTERNAL · USER-FACING · SCOPED

03 / TRADEOFFS

PROS & CONS

Four criteria. No spin. Pick what fits your threat model.

CRITERIA	API AUTHENTICATION	OAUTH AUTHENTICATION
⛨ SECURITY	✕ LOWER Static keys leak = full breach	✓ HIGHER Scoped, expiring, revocable
⚙ COMPLEXITY	✓ SIMPLE One header. One key. Done.	✕ COMPLEX Token dance, refresh, PKCE
↗ SCALABILITY	✕ LIMITED Key rotation = manual ops	✓ HIGH Per-user tokens, no shared secret
☻ USER EXPERIENCE	✕ MANUAL Each user manages own key	✓ SEAMLESS SSO via Google / GitHub

◆ THERE IS NO WINNER · ONLY THE RIGHT TOOL FOR THE RIGHT JOB

HOW OPENCLAW DOES IT

OPENCLAW · AUTH RUNTIME v2

OAUTH 2.0

STANDARD FOR 3RD-PARTY

IMPLEMENTATION HIGHLIGHTS

OAuth 2.0 for third-party integrations

Marketplace apps auth via PKCE-enabled authorization code flow.

API key auto-rotation

Internal keys rotate every 90 days via a single config flag — zero downtime.

Per-scope policy enforcement

OAuth tokens carry scoped claims enforced at the gateway — read, write, admin.

Full audit trail

Every auth event — key use, token grant, revocation — logged with trace ID.

05 / CONCLUSION

PICK THE RIGHT TOOL

API AUTH — INTERNAL USE

Best for trusted, internal, machine-to-machine flows where simplicity beats ceremony. Use it for CI runners, cron scripts, and service-to-service calls inside your VPC.

OAUTH — EXTERNAL ACCESS

Best for external, user-facing integrations where consent, scoping, and revocation matter. Use it for third-party apps, SSO dashboards, and any flow where a human is involved.

CALL_TO_ACTION.EXE

CHOOSE BASED ON SECURITY NEEDS & USE CASE.

There is no winner. API auth for internal use. OAuth for external / user-facing access. Match the method to your threat model.

INTERNAL → API

EXTERNAL → OAUTH

MIXED → BOTH

— END OF DECK —